UNIVERSIDAD TECNOLÓGICA NACIONAL

FACULTAD REGIONAL BUENOS AIRES

Práctica Profesional Supervisada

PLATAFORMA DE ENSAYOS DEL SISTEMA uFISIO

Nombre del Alumno: Hernán Matías Travado

Legajo Alumno: 131.453-1

Docente (Tutor Universitario): Sergio Díaz

Fecha de Entrega: 7 de julio de 2023

INFORME DEL SUPERVISOR DE CAMPO

El estudiante Hernán M. Travado llevó a cabo su Práctica Profesional Supervisada en el Grupo de Investigación y Desarrollo en Bioingeniería (GIBIO). El objetivo de la misma consistió en el desarrollo de una Plataforma Integral de Ensayos para el Sistema uFISIO, aplicado a mediciones cardiovasculares. Sin duda alguna, el desempeño de Hernán estuvo muy por encima de la media de las prácticas realizadas en el GIBIO, no sólo por su calidad profesional sino también por su capacidad para vincularse con los miembros de nuestro grupo. Más allá de haber cumplido con creces los objetivos planificados en su plan de trabajo, Hernán se desenvolvió con notable calidad profesional, mostrando gran iniciativa y una capacidad resolutiva admirable. Como resultado de ello contribuyó con mejoras no planificadas, incrementando de esta forma las capacidades del sistema implementado. Esto último demuestra el profundo nivel de compromiso alcanzado en el desarrollo de su práctica, habida cuenta de no contar, a priori, con conocimientos en el área cardiovascular.

Supervisor de Campo	**Responsable Secretaría de Ciencia, Tecnología e Innovación Productiva UTN.BA**
Firma 	**Firma** 
Aclaración **Leandro J. Cymberknop**	**Aclaración** **Patricia Cibeira**

Docente Supervisor
Firma
Aclaración

COMPROMISO DE AUTOR

Yo, Hernan Matías Travado, declaro que:

El contenido del presente documento es original y constituye un reflejo de mi trabajo personal. Manifiesto que, ante cualquier notificación de plagio, autoplagio, copia o falta a la fuente original, soy responsable directo legal, económico y administrativo sin afectar al Director del trabajo, a la Universidad y a cuantas instituciones hayan colaborado en dicho trabajo, asumiendo las consecuencias derivadas de tales prácticas.

Firma:

Aclaración: Hernan M. Travado

Fecha: 7 de julio de 2023

ÍNDICE GENERAL

1. Introducción 4
2. Objetivo 4
3. Desarrollo de la plataforma 5
    1. Desarrollo del Hardware 5
        1. Diagrama Esquemático 5
        2. Diseño del PCB 7
        3. Montaje de los componentes 8
        4. Configuración del Microcontrolador 9
    2. Sistema Operativo 12
    3. Implementación del protocolo de comunicación 17
    4. Detalles de las tareas 19
        1. Tarea de lectura de presión 19
        2. Tarea de lectura del sensor PPG 19
        3. Tarea de lectura de distancia 20
        4. Tarea de análisis de datos 20
        5. Tarea de visualización 21
    5. Uso de la memoria del microcontrolador 22
    6. Clientes de Ejemplo 22
4. Resultados 22
    1. Detalle de la placa 22
    2. Utilizando el Software provisto por el GIBIO 24
    3. Utilizando el cliente de pruebas 25
5. Conclusiones 27
6. Referencias 27

1. Introducción

Las enfermedades cardiovasculares son un grupo de trastornos que incluye varias enfermedades del corazón y del sistema circulatorio. La Organización Mundial de la Salud se refiere a las ECV como la principal causa de muerte en todo el mundo, lo que resulta en un número de personas fallecidas anualmente más que por cualquier otra causa. Las mejoras de diagnósticos, tratamientos, medicamentos, y las técnicas quirúrgicas fueron responsables de una importante disminución de la mortalidad total por ECV en las últimas décadas. La detección temprana basada en las condiciones patologías es la clave para la supervivencia del paciente. Existen diversos métodos para la evaluación de pacientes con factores de riesgo cardiovascular, pero ninguno es basado en una sola medición. La velocidad de onda de pulso (VOP) es una medida indirecta bien establecida de la rigidez arterial. La onda de pulso es transmitida a través del sistema arterial, y su velocidad esta inversamente relacionado con la flexibilidad de las paredes de las arterias. Se puede determinar la VOP realizando la medición del desfasaje temporal entre dos ondas de pulso en zonas distintas del sistema arterial. Particularmente el valor medido entre las arterias carótidas y femoral (VOP_{CF}) ha demostrado ser un predictor de eventos cardiovasculares.

2. Objetivo

El objetivo del presente trabajo es desarrollar una plataforma de pruebas para el sistema uFISIO desarrollado por el Grupo de Investigación y Desarrollo en Bioingeniería (GIBIO). Este dispositivo consiste en realizar la medición simultanea de la onda de pulso en la arteria carótida y femoral y mediante un algoritmo de procesamiento de señales determinar el desfasaje temporal entre ambas señales de pulso. Dado que este sistema es experimental y se requiere cierta versatilidad en la utilización de diversos sensores y algoritmos de procesamiento es que se desarrolló esta plataforma. Las premisas de desarrollo son el enfoque en la versatilidad en el uso de diferentes sensores, ya sean sensores tonométricos o fotopletismográficos, diseñar un sistema operativo robusto que permita el procesamiento de la señal dentro del dispositivo, la implementación de un protocolo de comunicación que permita comunicar de manera universal no solo este equipo sino los posibles futuros desarrollos y la posibilidad de tener un método de visualización de los resultados en el mismo equipo. Para cumplir con estas premisas se desarrolló un PCB que incluye los siguientes periféricos: Un módulo de microcontrolador basado en el STM32F104C8T6 conocido vulgarmente como la “Blue Pill”, un módulo de adquisición de señales de alta precisión basado en el conversor analógico-digital ADS1192 que puede ser utilizado para la adquisición tanto de señales de ECG como señales producidas por sensores tonométricos piezoeléctricos, un sensor fotopletismográfico basado en el MAX30102 que cuenta con un método de adquisición mediante reflexión de luz roja e infrarroja, una pantalla OLED de 128x64 px y un módulo de conexión bluetooth HC-05 para él envió de datos. Además, se implementó una fuente linear de 3.3v para la alimentación de los distintos periféricos, se dejó disponible una conexión serial para evitar el uso del módulo bluetooth, se instalaron 2 leds indicadores y además se dejaron 4 entradas digitales las cuales serán usadas para la configuración de la plataforma.

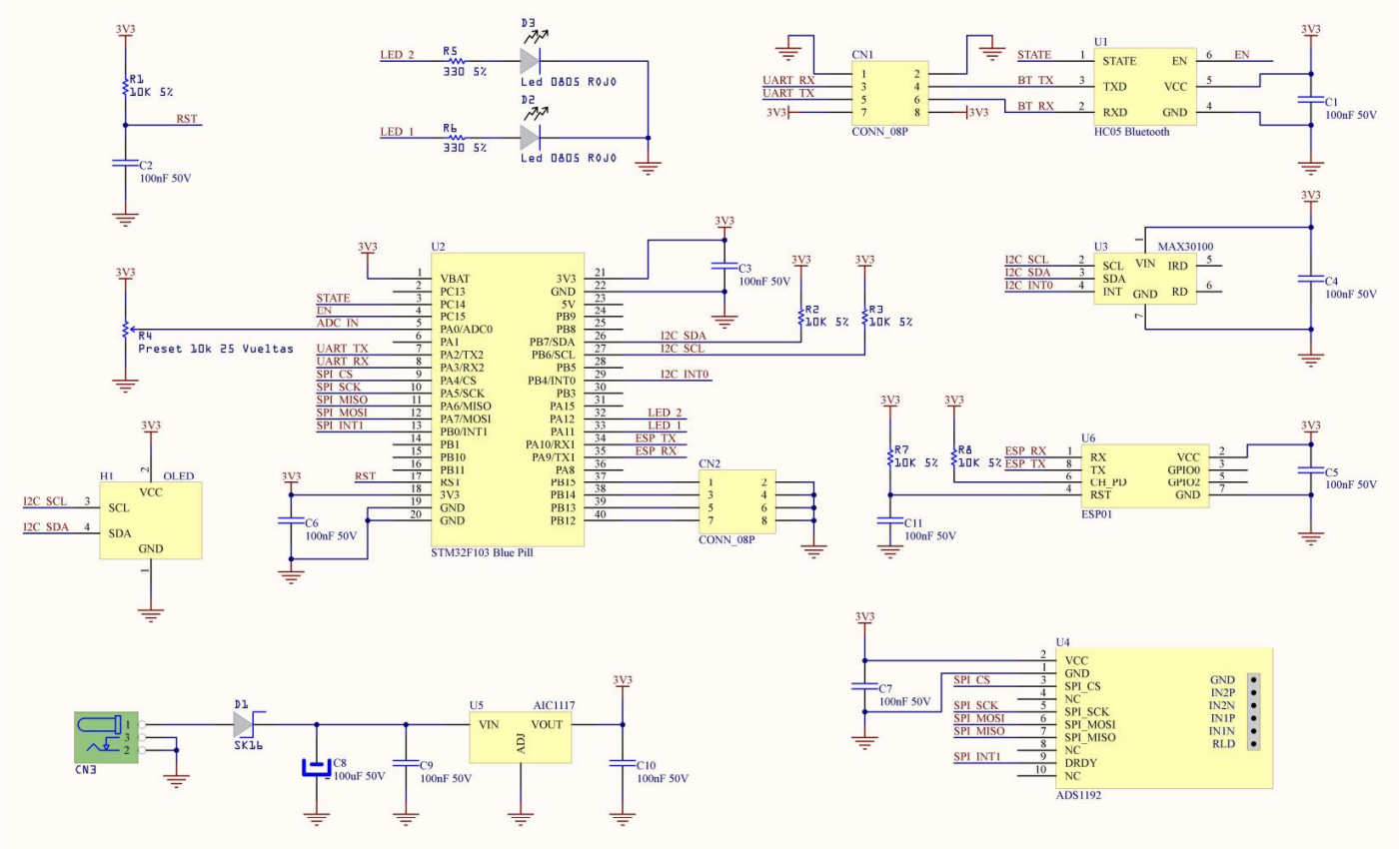
3. Desarrollo de la plataforma

1. Desarrollo del Hardware

El diseño del pcb se realizo en Altium Designer v22.11. Los archivos de diseño se encuentran en la carpeta “PCB Plataforma de Pruebas”.

1. Diagrama Esquemático

Dadas las premisas de implementación se decidió desarrollar la plataforma con el siguiente diagrama esquemático.



Podemos observar una fuente lineal de 3.3v basada en el regulador de voltaje AIC1117 con sus correspondientes capacitores de filtrado. El módulo microcontrolador basado en el STM32F104C8T6 con sus respectivos filtros de alimentación y línea de reset acoplada con un capacitor y una resistencia. El puerto I2C1 se conecta a la pantalla OLED de 1.3” y al módulo con

el sensor fotopletismográfico MAX30102, además se conecta la interrupción del MAX30102 a la entrada PB4/INT0 del microcontrolador para disparar la rutina de lectura del dato del sensor.

Con respecto a la comunicación se dispone a conectar un módulo wifi basado en el ESP01 al puerto UART1 del microcontrolador con su respectivo filtro de fuente y se conecta el módulo bluetooth HC-05 al puerto UART2 del microcontrolador. El módulo bluetooth no se conectó directamente al puerto serial, sino que se conectó a través del conector CN1, para la utilización del módulo bluetooth hay que colocar dos jumpers en las líneas TX y RX. Se implementó de esta forma por dos motivos, primero de esta manera levantando los jumpers se puede conectar un adaptador USB-TTL para conectar el dispositivo directamente al USB. Por otro lado, al levantar los jumpers se puede conectar un adaptador USB-TTL directamente al módulo bluetooth para poder configurar el mismo. Recordamos que el módulo bluetooth por defecto tiene una velocidad de transmisión de datos de 38400 bps la cual debe ser modificada a 115200 bps. Existe en el mercado un módulo el cual se puede configurar directamente desde el microcontrolador ya que dispone una entrada para seleccionar el módulo en modo configuración, el módulo que se dispone no tiene esta entrada, sino que tiene un pulsador el cual hay que mantener presionado durante el arranque para entrar en modo programación.

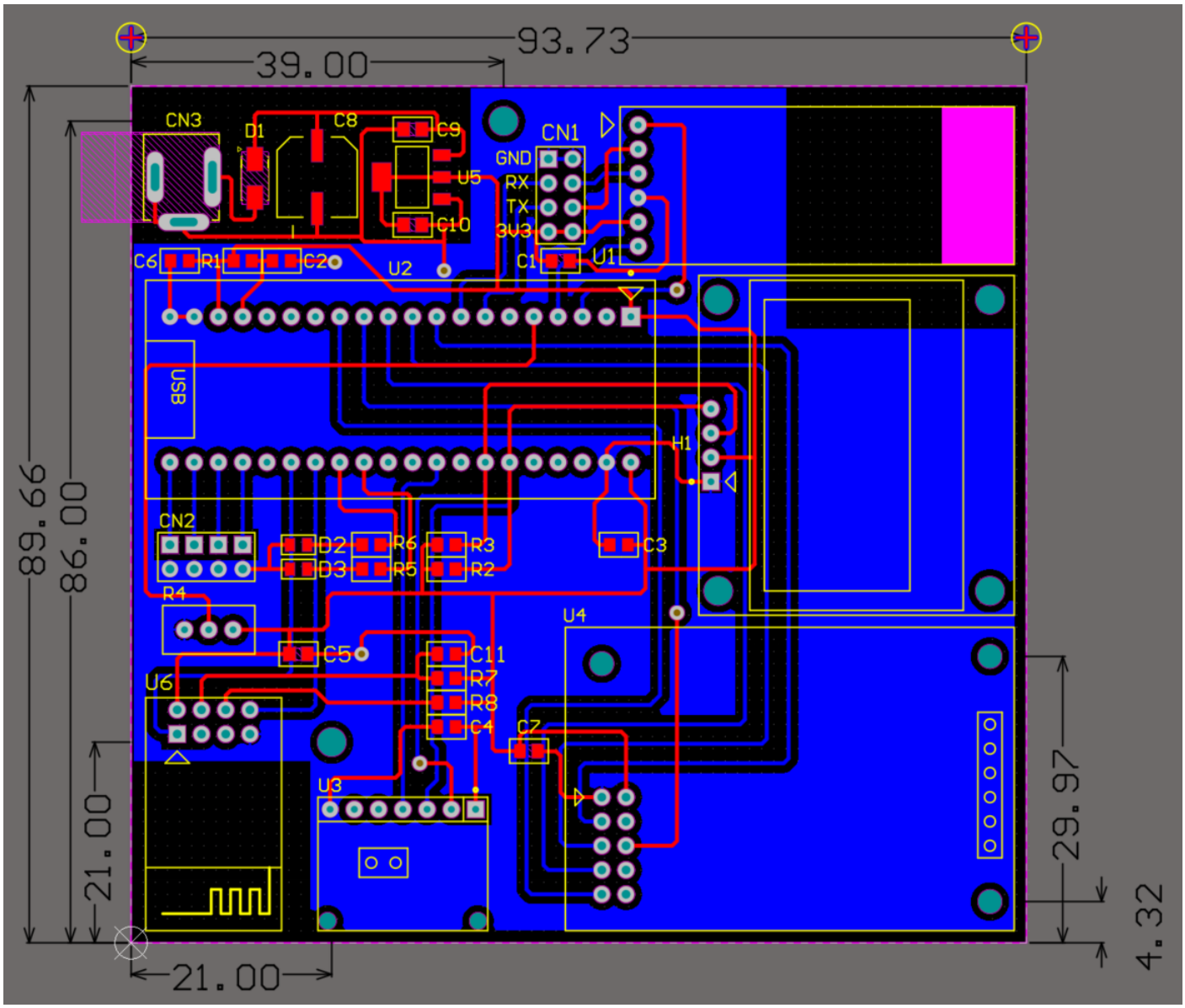
Se conecto el módulo ADS1192 al puerto SPI1 del microcontrolador utilizando el pin A4 como chip select y además se conectó la entrada de interrupción PB0/INT1 a la salida DRDY (Data Ready) del conversor analógico digital. Mediante la lectura de esta interrupción se lanza la lectura del dato disponible en el puerto SPI.

Se utilizan los pines PB12-PB15 como entradas de uso general que durante el desarrollo del software se van a utilizar para configurar el sistema. Estas entradas digitales deben estar configuradas como entradas con pull up interno, ya que su activación va a ser mediante jumpers a GND. Además, se conectan dos LEDs indicadores a los pines PA11 y PA12, es por esto que estos pines deben ser configurados como salida push pull.

Por último, se utilizó el pin PA0 como entrada del canal 0 del conversor analógico digital del microcontrolador. El sistema completo prevé la medición de la distancia carótida-femoral, esta medición se hace mediante un sistema de cordón retráctil y el avance de este cordón se mide con la rotación de un potenciómetro multivuelta, la variación de esta resistencia es utilizada como entrada analógica e interpretada por el microcontrolador. Como al momento del desarrollo de este proyecto no hay precisiones de cómo es el sistema de medición se dejó cableado un preset multivuelta el cual simula el comportamiento de este dispositivo.

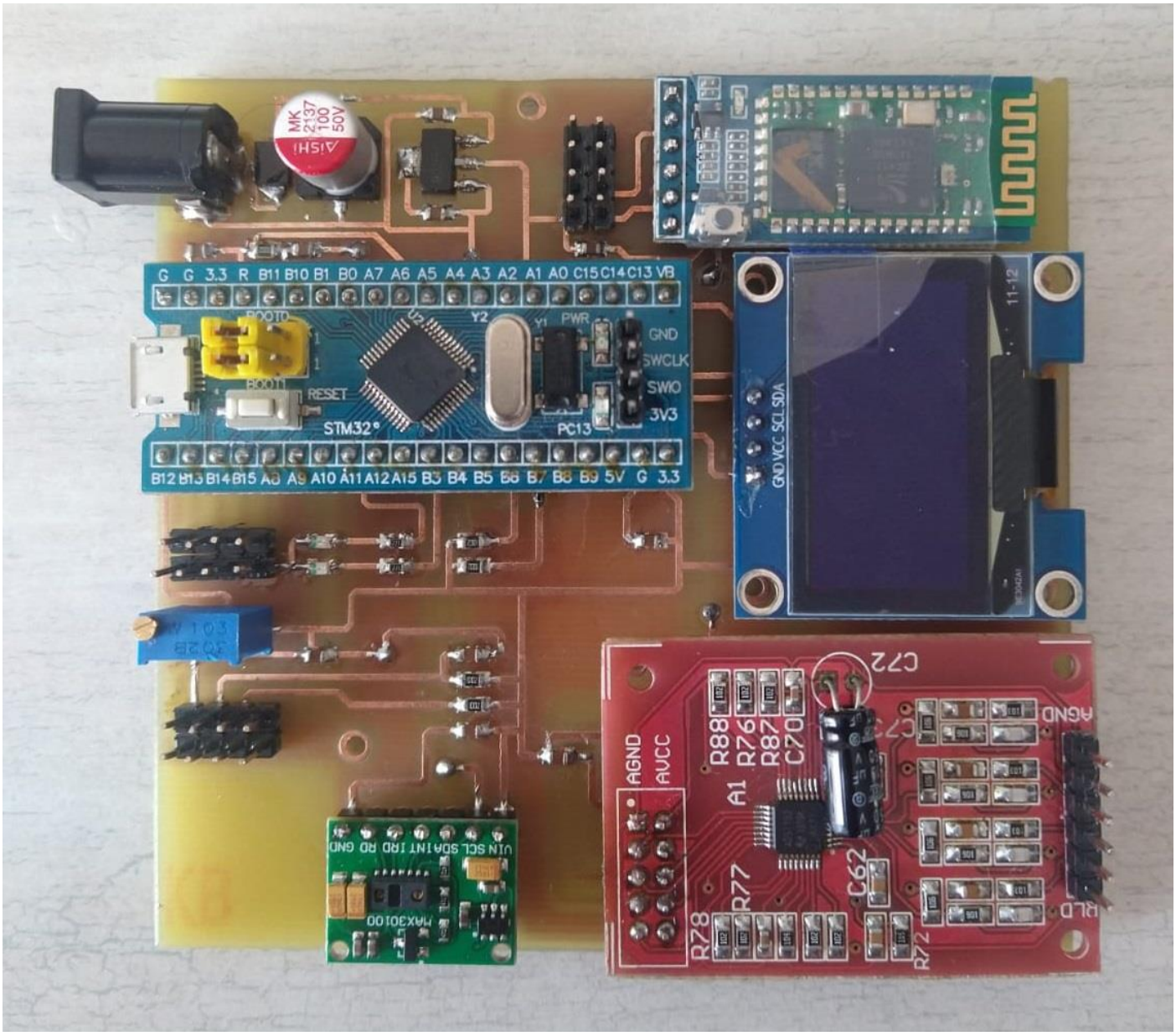
2. Diseño del PCB

En base al esquemático propuesto se realizó el PCB.



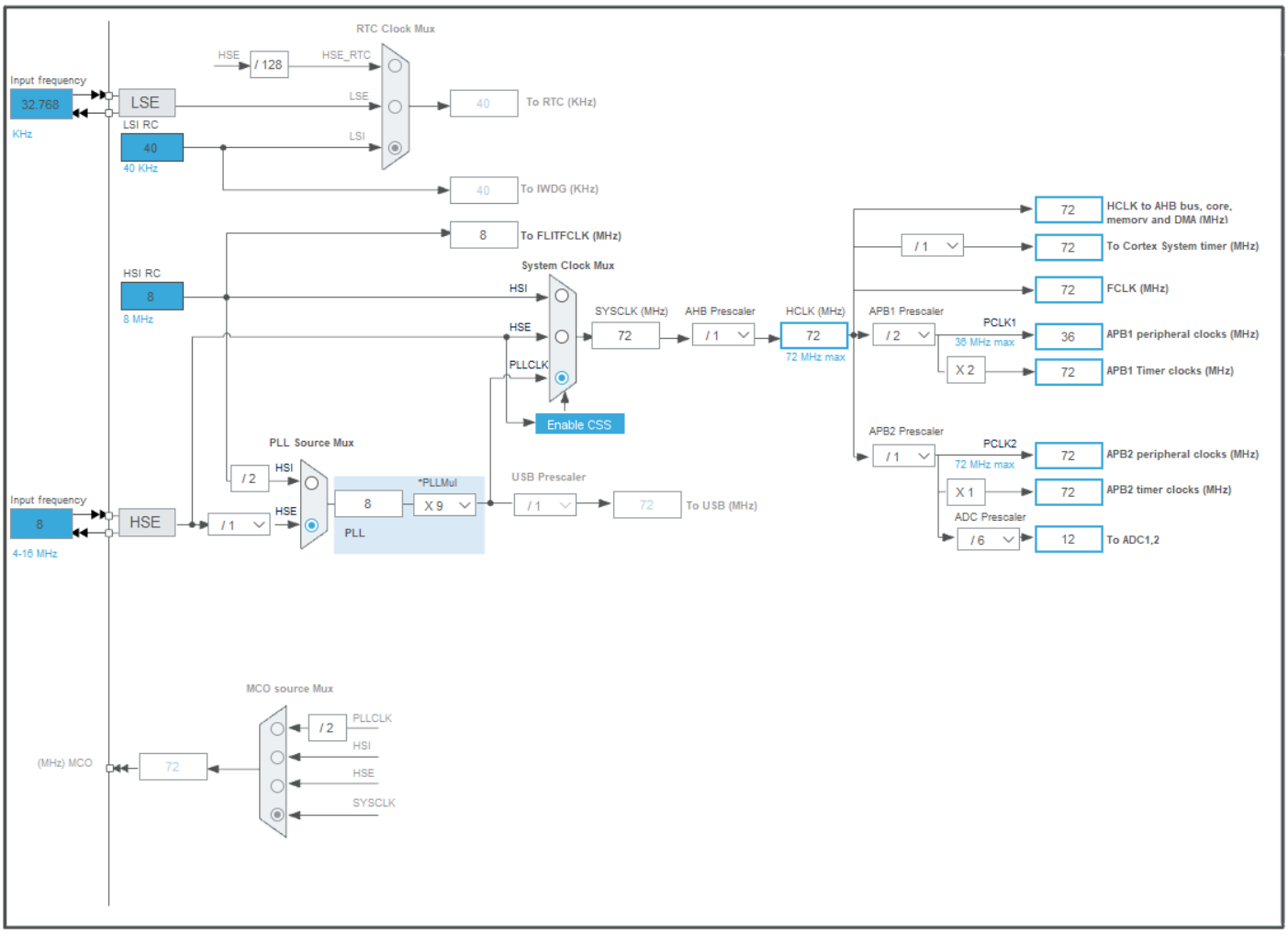
3. Montaje de los componentes

Luego se implementó y se montaron los componentes.



4. Configuración del Microcontrolador

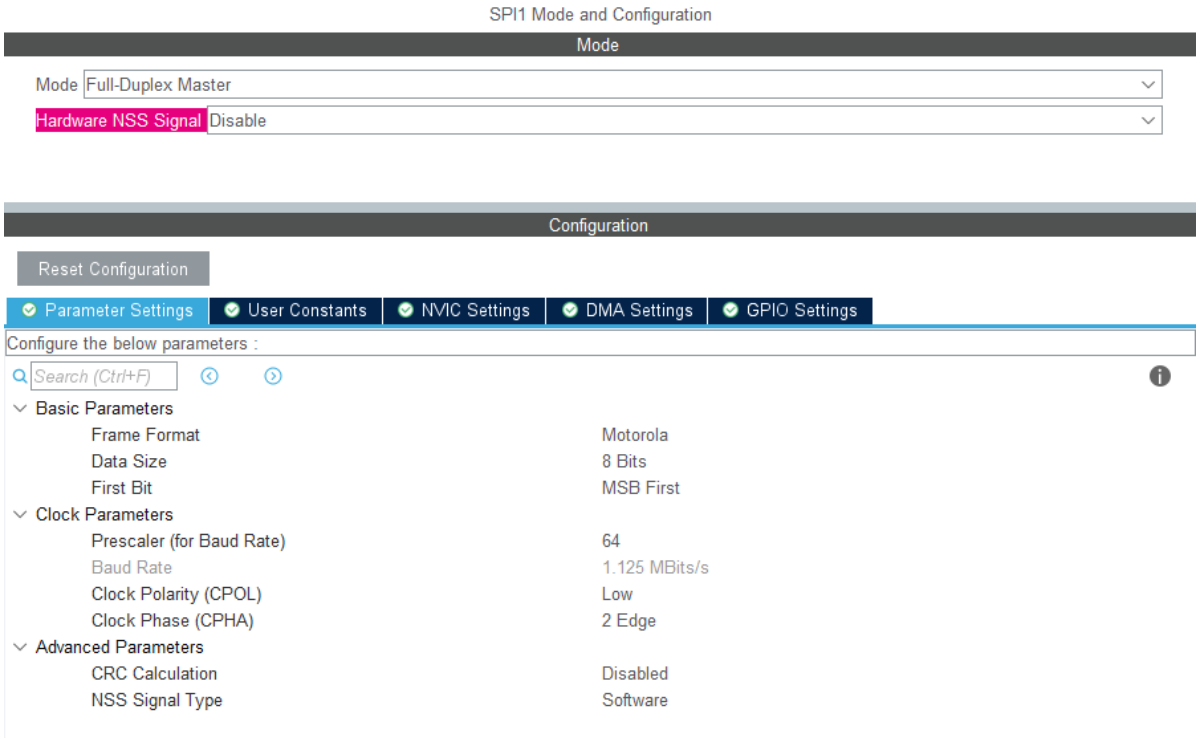
Configuración del clock:



Configuración de los pines GPIO:

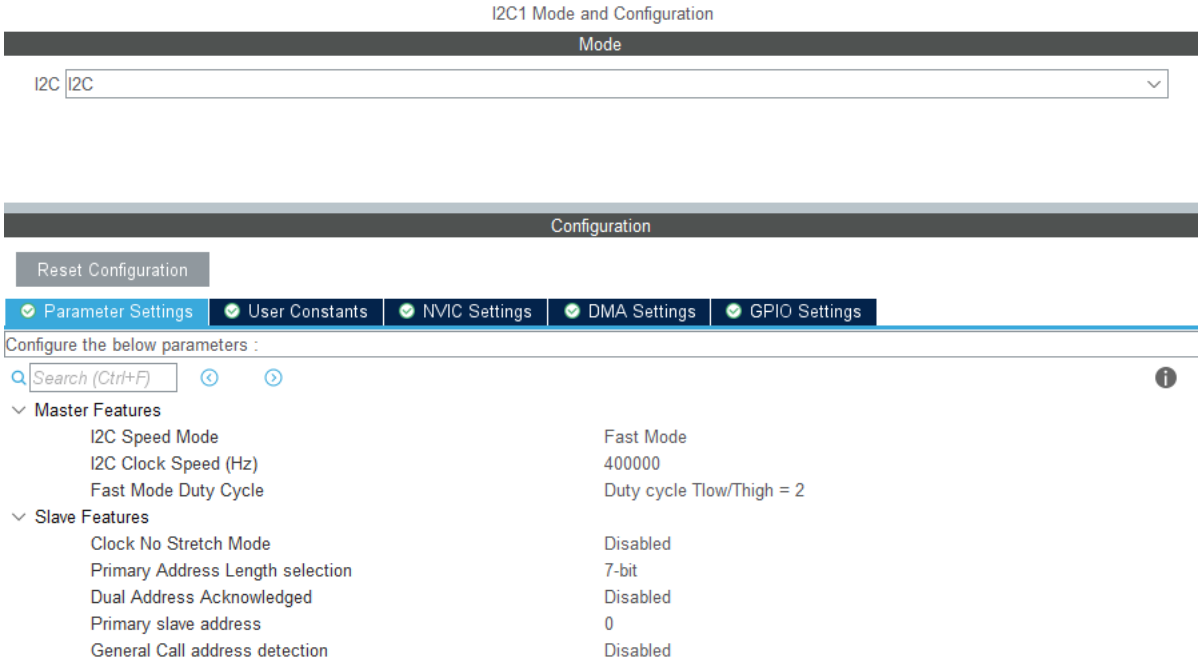
Pin Name	Signal on Pin	GPIO output level	GPIO mode	GPIO Pull-up/Pull-down	Maximum output speed	User Label
PA1	n/a	n/a	Input mode	No pull-up and no pull-down	n/a	
PA4	n/a	Low	Output Push Pull	No pull-up and no pull-down	Low	SPI_CS
PA11	n/a	Low	Output Push Pull	No pull-up and no pull-down	Low	LED_1
PA12	n/a	Low	Output Push Pull	No pull-up and no pull-down	Low	LED_2
PB0	n/a	n/a	External Interrupt Mode with Falling edge trigger detection	Pull-up	n/a	
PB4	n/a	n/a	External Interrupt Mode with Falling edge trigger detection	Pull-up	n/a	
PB12	n/a	n/a	Input mode	Pull-up	n/a	GEN_IN_1
PB13	n/a	n/a	Input mode	Pull-up	n/a	GEN_IN_2
PB14	n/a	n/a	Input mode	Pull-up	n/a	GEN_IN_3
PB15	n/a	n/a	Input mode	Pull-up	n/a	GEN_IN_4
PC13-TAMPER-RTC	n/a	Low	Output Push Pull	No pull-up and no pull-down	Low	BUILTIN_LED

Configuración del SPI1:

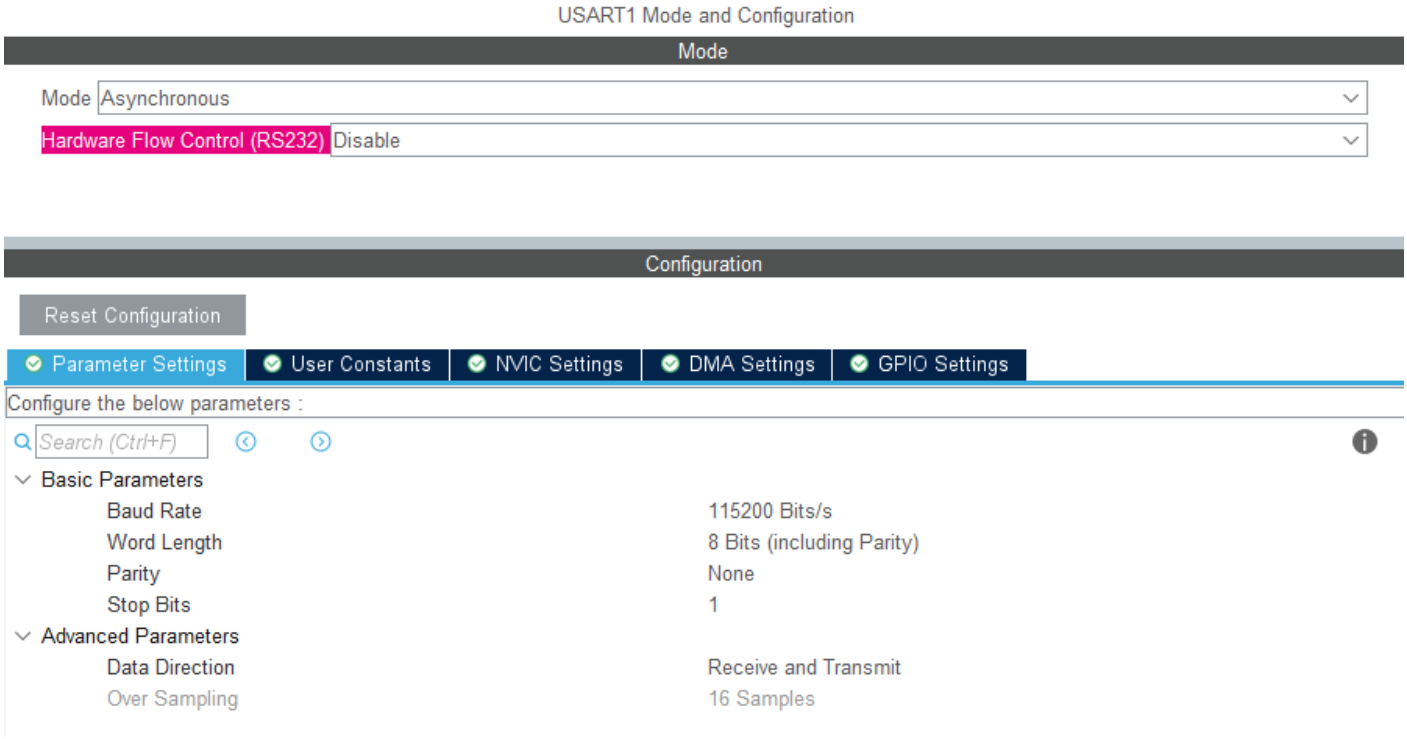


Se selecciona el Clock Polarity en LOW y el Clock Phase en 2 Edge, y se ajusta el prescaler en 64 para seleccionar la velocidad de transmisión en 1.125 MBits/seg.

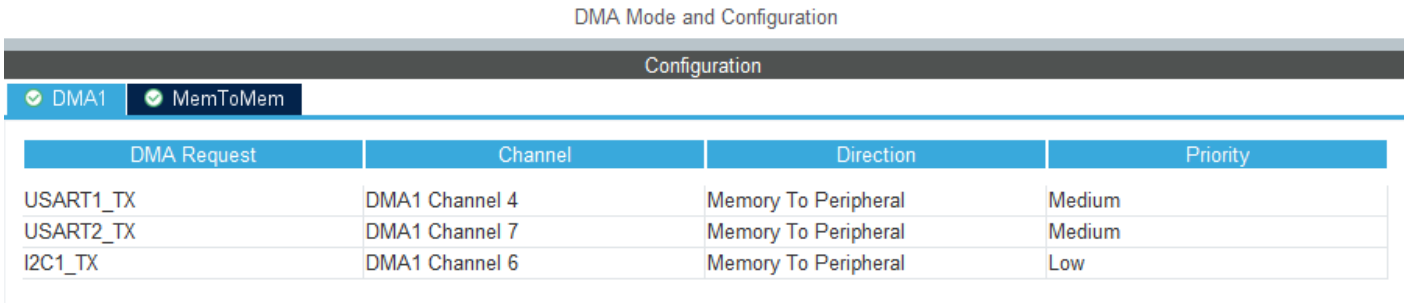
Configuración del I2C1:



Configuración del puerto UART1 y UART2



Configuración del DMA:



2. Sistema Operativo

Se decidió implementar un sistema operativo con enfoque en tiempo real. Se adaptaron algunos conceptos de FreeRTOS implementados a menor escala. Dado que sistema se basa en la adquisición de datos y como consenso se utilizó una frecuencia de muestreo de 1 kHz se decidió establecer un Tick de sistema cada 1 ms. Si bien la adquisición de datos no se haga de manera sincrónica, es necesario este Tick para marcar el ritmo del sistema. Es decir, independientemente del método de adquisición, cada 1 ms habrá una muestra nueva disponible para ser procesada y eventualmente enviada al cliente.

Se separaron las tareas en dos grupos. El primer grupo son tareas que, si o si tienen que realizarse en cada ciclo de ejecución, es decir, estas tareas hacen un uso apropiativo del CPU. El segundo grupo son tareas que no necesariamente tiene que terminar su ejecución en el mismo ciclo, en este grupo están las tareas que se encargan del procesamiento de las señales, estadísticas del sistema, visualización local de los resultados, etc. Estas tareas son de menor relevancia en el sistema y por lo tanto se ejecutan en el tiempo restante luego de que se ejecutan las tareas apropiativas.

Las tareas que van a hacer un uso apropiativo del CPU tienen que empezar y terminar en un return, no deben hacer uso excesivo del CPU ni realizar demoras o esperas activas. La estabilidad del sistema depende que ninguna tarea se exceda en el tiempo de ejecución. Por este motivo se implementó un sistema de estadística de uso del CPU por cada tarea el cual se maneja dentro del scheduler. En cambio, para las tareas no apropiativas, la ejecución tiene que hacerse dentro de un bucle while infinito ya que el scheduler va a sacar de contexto esta tarea cuando sea el momento de ejecutar las tareas apropiativa resguardando dicho contexto para que la tarea pueda ser resumida cuando sea su turno de ejecución.

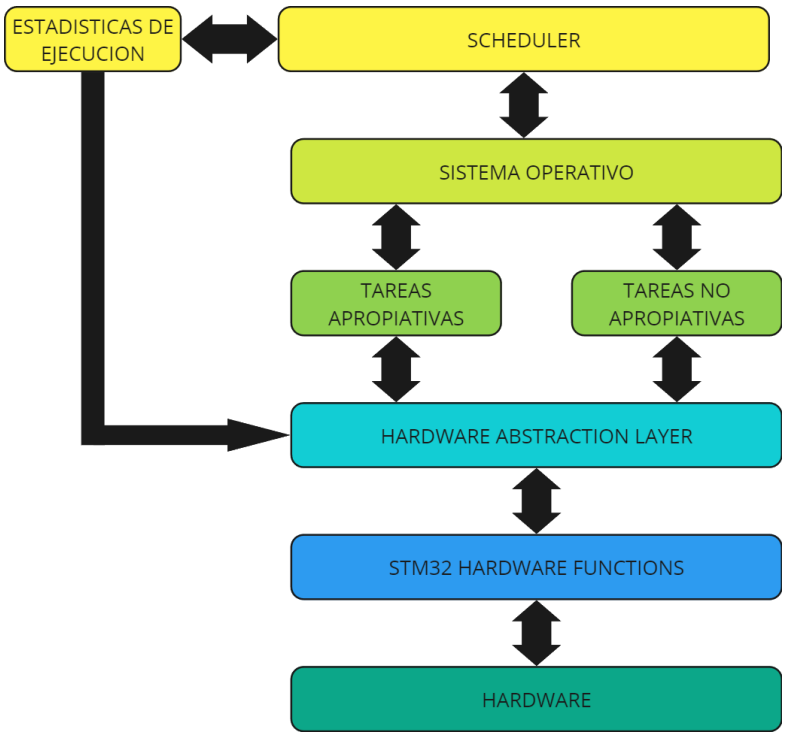
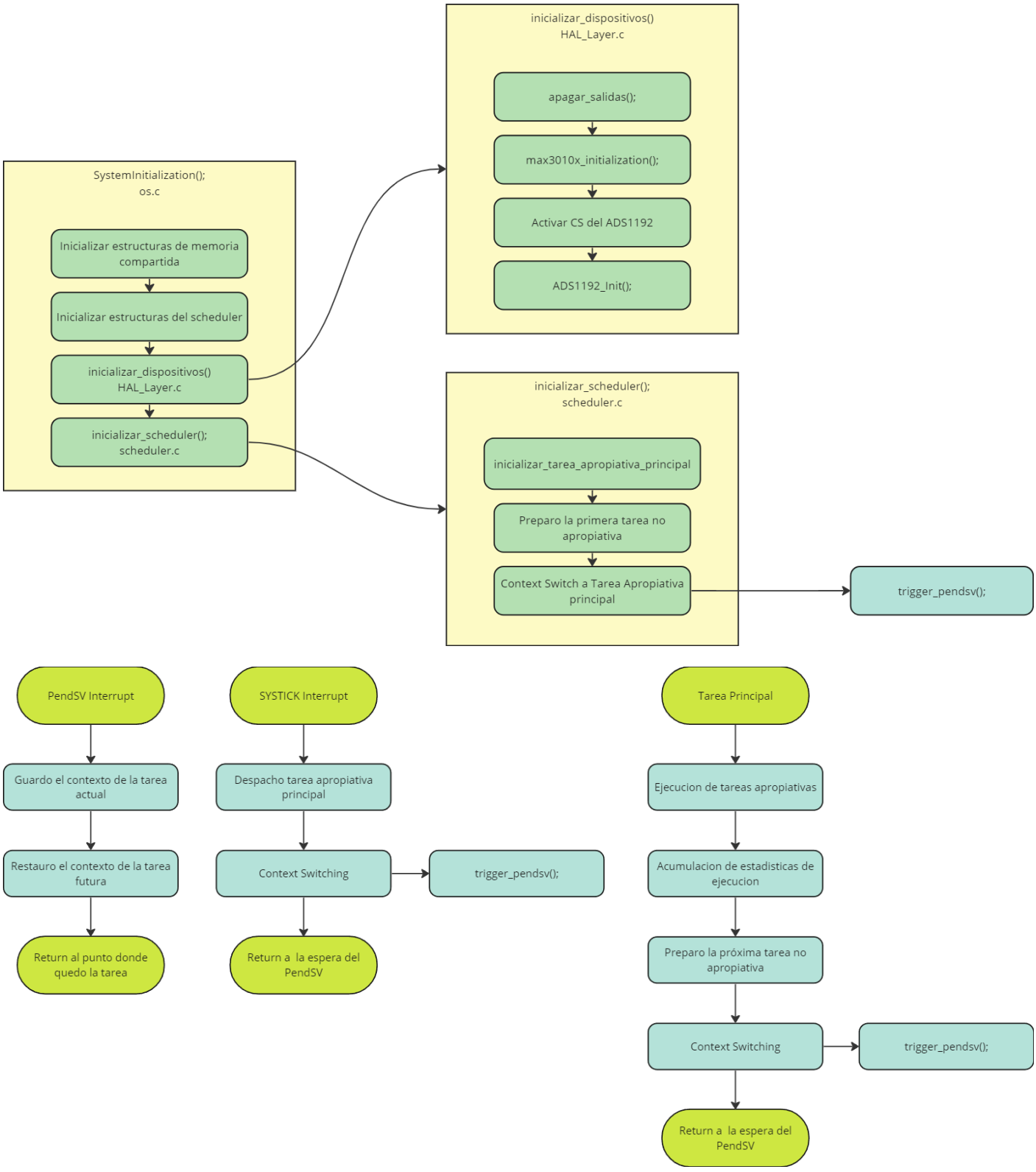
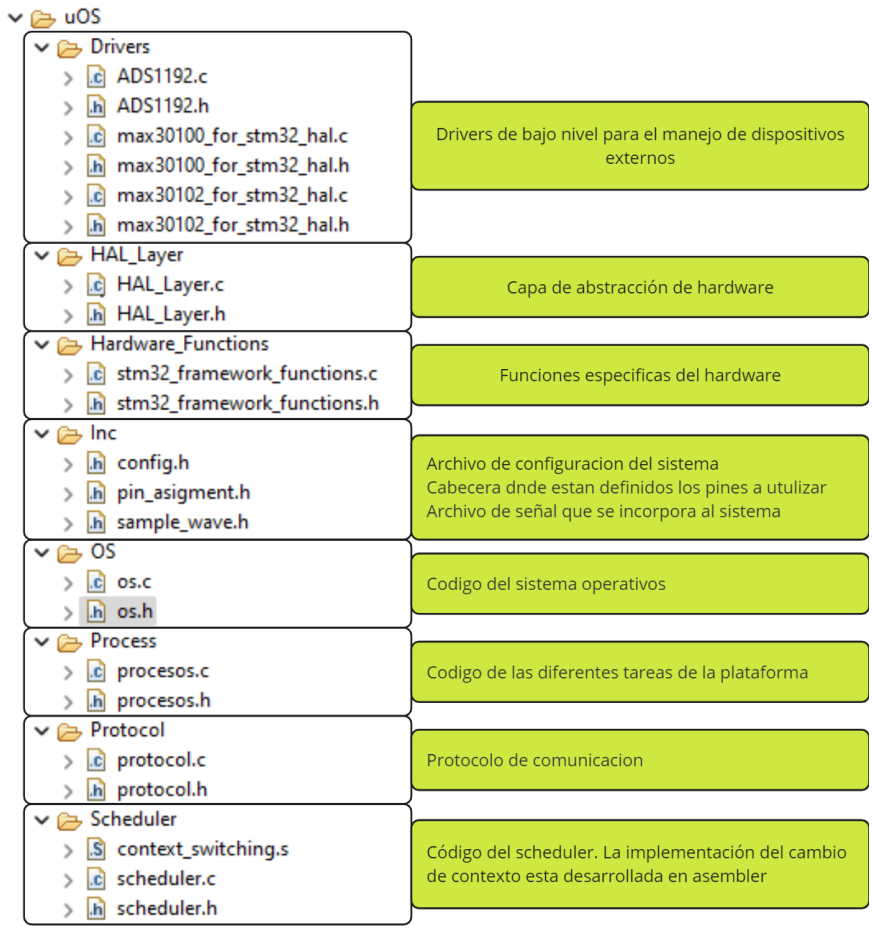


Diagrama de flujo de ejecución:



Estructura de Archivos:



Identificación de los pines (pin_asigment.h):

```
/*
 * pin_asigment.h
 *
 *  Created on: 14 mar. 2023
 *      Author: Hernan Travado
 */

#ifndef UOS_INC_PIN_ASIGMENT_H_
#define UOS_INC_PIN_ASIGMENT_H_

#include "Inc/config.h"


extern ADC_HandleTypeDef hadc1;
extern I2C_HandleTypeDef hi2c1;
extern SPI_HandleTypeDef hspi1;
extern TIM_HandleTypeDef htim2;
extern UART_HandleTypeDef huart1;
extern UART_HandleTypeDef huart2;

/* Led integrado */
#define BUILT_IN_LED_PORT       GPIOC
#define BUILT_IN_LED_PIN        GPIO_PIN_13

/* Built-in Inputs */
#define GENERIC_INPUTS_PORT     GPIOB
#define PROTOCOL_SELECTION_PIN  GPIO_PIN_12
#define SIGNAL_SOURCE_PIN       GPIO_PIN_13
#define INTERFACE_SELECTION_PIN GPIO_PIN_14
#define GEN_INPUT_4             GPIO_PIN_15

/* On Board LEDs */
#define ONBOARD_LEDS_PORT       GPIOA
#define LED_1_PIN               GPIO_PIN_12
#define LED_2_PIN               GPIO_PIN_11

/* Comunication Handlers Defines */
#define BLUETOOTH_UART          &huart2
#define SERIAL_COM_UART         &huart2
#define ESP8266_UART            &huart1
#define DIST_ADC_HANDLER        &hadc1
#define SpO2_HANDLER            &hi2c1
#define OLED_HANDLER            &hi2c1
#define SPI_HANDLER             &hspi1

/* ADS PINs */
#define ADS_CS_PIN              GPIO_PIN_4
#define ADS_CS_PORT             GPIOA
#define ADS_DRDY_PIN            GPIO_PIN_0
#define ADS_DRDY_PORT           GPIOB

/* I2C Address */
#define OLED_ADDR               0x3C
#define SpO2_ADDR               0x57

/* Distancia ADC Channel */
#define DIST_ADC_CHANNEL        0

#endif /* UOS_INC_PIN_ASIGMENT_H_ */
```

Enlaces a los dispositivos externos para los defines

LED Integrado en el modulo Blue Pill

Identificación de las entradas genéricas

LEDs integrados en la placa

Handlers de comunicación

Pines especificos para el control del ADS1192

Direcciones de I2C

Canal analógico para medir la distancia

Configuración del Sistema (config.h):

```
/*
 * Tamaño de la memoria compartida para el almacenamiento de los datos
 * El buffer va a ser del tipo cola circular
 */
#define SHARED_MEMORY_SIZE 2048

/* Cantidad de tareas que puede manejar el planificador
 * Setear este valor lo mas bajo posible para optimizar el uso de la RAM
 */
#define MAX_LEN_TASK_LIST 4

/* Cantidad de muestras para el filto de media movil para la medicion de la distancia. */
#define CANTIDAD_MUESTRAS_DISTANCIA 50
#define PERIODO_REPORTE_DISTANCIA_ms 50
#define FACTOR_DE_CONVERSION_DIST 1.0
#define DISTANCIA_MINIMA 20         // Distancia en cm para el valor minimo del adc
#define DISTANCIA_MAXIMA 120        // Distancia en cm para el valor maximo del adc

/* Intervalo de reporte del uso del CPU en milisegundos */
#define CPU_REPORT_INTERVAL_ms 5000

/* Longitud del stack */
#define STACK_SIZE 1024

/* Defino si uso el MAX30100 o el MAX30102. EL MAX3010x se refiere a la libreria de arduino adaptada a STM32
 * NO PUEDO USAR LOS DOS AL MISMO TIEMPO, TIENE QUE HABER UN SOLO DEFINE*/
//#define USE_MAX30100 1
//#define USE_MAX30102 1
#define USE_MAX3010x 1

// Configuracion del MAX3010x. Esta config solo tiene efecto si se usa el MAX3010x
#define PPG_SAMPLE_RATE 100 //Options: 50, 100, 200, 400, 800, 1000, 1600, 3200
#define PPG_PULSE_WIDTH 118  //Options: 69, 118, 215, 411
#define PPG_AVG_COUNT 16         //Options: 1, 2, 4, 8, 16, 32
#define PPG_LED_CURRENT 0x3F //Options: 0=Off to 255=50mA
#define PPG_ADC_RANGE 4096   //Options: 2048, 4096, 8192, 16384

/*   PPG Allowed Setting
 *
 *      SPS |     PULSE WIDTH
 *          | 69 | 118 | 215 | 411 |
 *   -------+----+-----+-----+-----+
 *       50 | OK | OK  | OK  | OK  |
 *      100 | OK | OK  | OK  | OK  |
 *      200 | OK | OK  | OK  | OK  |
 *      400 | OK | OK  | OK  | OK  |
 *      800 | OK | OK  | OK  |     |
 *     1000 | OK | OK  |     |     |
 *     1600 | OK |     |     |     |
 *     3200 |    |     |     |     |
 *   -------+----+-----+-----+-----+
 *  ADC RES | 15 | 16  | 17  | 18  |
 *  --------+----+-----+-----+-----+
 */

/* Defino si uso el ADS1192 */
#define USE_ADS1192 1

/* Defino si uso el display OLED */
#define USE_OLED_I2C 1

/* Periodo de actualizacion del display OLED */
/* Tener en cuenta que la transmision de los datos de actualizacion demora alrededor de 25 ms  */
#define UPDATE_SCREEN_PERIOD_ms 100

#define SHOW_SIGNAL_CAROTIDA_DURATION 2000 // Cantidad de ms que muestro la señal en pantalla
#define SHOW_SIGNAL_FEMORAL_DURATION 2000  // Cantidad de ms que muestro la señal en pantalla
#define SHOW_STAT_DURATION 5000            // Cantidad de ms que muestro las estadisticas en pantalla

#endif /* UOS_INC_CONFIG_H_ */
```

Funciones de la capa de abstracción de hardware

Se implementan dentro del archivo HAL_Layer.c, estas funciones son una capa intermedia entre el código del sistema operativo y el hardware. Cuestiones específicas del manejo de los dispositivos están desarrolladas en este archivo. A si mismo, las funciones específicas del hardware del STM32 están implementadas en el archivo stm32_framework_functions.c, por lo tanto, si se quisiera implementar el dispositivo en otro hardware solo habría que reescribir las funciones que se encuentran en este archivo.

3. Implementación del protocolo de comunicación

El protocolo de comunicación se encuentra implementado en los archivos protocol.c y protocol.h. En el archivo de cabecera se encuentran declaradas todas las estructuras de todos los tipos de datos que se van a transmitir por el puerto. Al momento de implementar el cliente se sugiere respetar estas declaraciones para evitar problemas de alineamiento de datos. Estas estructuras se van a serializar y enviar por el puerto, en el cliente podrían ser implementado objetos con la misma estructura y deserializarlos. Dentro del repositorio se encuentra ejemplos de implementación de clases tanto en Python como en C++.

Para el caso de la presión y sensor fotopletismográfico se acumulan las muestras y se envían en paquetes de 32 muestras de esta manera se optimiza el uso del puerto evitando overhead innecesario. En el caso de la presión es posible que las mediciones no se realicen exactamente en el mismo instante, incluso se podrían usar dos métodos de medición distintos. Es por esto por lo que hay implementada dos funciones, una StoreFemoralPressure y la otra StoreCarotidPressure, estas funciones almacenan los datos de presión es una variable estática y cuando ambas señales completan las 32 muestras se procede a enviar por el puerto a través de la función SendPressure. Este comportamiento es automático, el usuario lo único que tiene que preocuparse es de llamar a las funciones de Store desde la tarea y procurar que ambas señales tengan el mismo periodo de muestreo. En el caso del sensor fotopletismográfico hay una función que recolecta las muestras StoreSpO2Data en este caso esta función recibe tanto el dato del sensor infrarrojo como el del sensor rojo y una vez que completa el set de 32 muestras se envía por el puerto con la función SendSpO2Data.

El resto de las funciones que envían datos por el puerto no utilizan ningún mecanismo de acumulación de datos, las funciones son las siguientes: SendVOPandHBR recibe como parámetro la VOP calculada y el periodo de la última pulsación detectada. Este dato se envía cada vez que se detecta un latido. SendCPUData esta función recibe como parámetro los tiempos de ejecución de las tareas y lo envía al cliente, el periodo de este reporte se configura desde el archivo config.h. SendDistance envía la distancia medida por el conversor ADC interno del microcontrolador, las muestras se toman en todos los ticks del sistema y el periodo de envío se define en el archivo config.h Cabe destacar que la versión original del protocolo solo prevé el envío de datos de presión, es por esto que la única función que envía datos cuando esta seleccionada la versión 2 del protocolo es la función que reporta la presión, las demás funciones solo reportan cuando esta seleccionada la versión 3 del protocolo.

El protocolo de comunicación está formado por los siguientes elementos:

HEADER								PAYLOAD
SYNC				COMANDO	NONCE	CHECKSUM	PAYLOAD LENGTH	Custom Data
0x23	0x23	0x13	0x10	uint32	uint32	uint16	uint16	

La cabecera está compuesta por 4 bytes de sincronismo, se utilizaron los mismos del protocolo versión 2. Seguido tenemos un entero de 32 bits que representa un comando o un tipo de paquete. Los posibles comandos se definieron en un enum en el archivo protocol.h

```
enum CMD_TYPES {
	BAD_CMD = 0,
	ACK,
	uFISIO_PRESSURE_DATA,
	uFISIO_DISTANCE,
	uFISIO_SpO2_DATA,
	uFISIO_VOP_HBR,
	uFISIO_CPU_REPORT

};
```

Dado que el protocolo se pensó para que sea universal y que varias plataformas puedan compartir el mismo protocolo, en un futuros esta lista de comandos podría incrementarse y así poder manejar otro tipo de paquetes. Los paquetes con comando igual a cero deberían ser descartados por los clientes.

El nonce es un entero de 32 bits que se incrementa en cada paquete, cuando el equipo enciende empieza con el nonce=1 y de ahí en más lo incrementa con cada paquete enviado, de esta manera en el cliente se puede tener seguimiento de los paquetes recibidos y así poder detectar si hay algún paquete perdido, en futuras versiones se prevé la retransmisión de paquetes a pedido del cliente.

El campo checksum es un entero de 16 bits que se utiliza para verificar la integridad del paquete recibido, el cálculo del checksum se hace únicamente sobre el payload, la cabecera no se incluye en este checksum y se realiza haciendo la suma byte a byte de todo el payload haciéndolo desbordar en un entero de 16 bits. La función que calcula el checksum es la siguiente:

```
32 /**
33   * @brief  Funcion que calcula el checksum del payload
34   * @param  uint8_t    *p: Puntero al comienzo del payload
35   * @param  uint16_t size: Tamaño del payload en bytes
36   * @retval uint16_t Resultado de la suma byte a byte
37   */
38 uint16_t calculateChecksum(uint8_t *p, uint16_t size){
39     uint16_t ret=0;
40     for(uint16_t i=0; i<size; i++){
41         ret += *p;
42         p++;
43     }
44     return ret;
45 }
```

El campo payload length como su nombre lo indica es el largo total en bytes del payload, está representado por un entero de 16 bits por lo tanto teóricamente el payload máximo que se puede transmitir es de 64kbytes.

A continuación de la cabecera se envía el payload, el mecanismo de recepción de este payload dependerá del tipo de dato que se esté mandando. La identificación de este paquete se hace mediante el campo comando. Cada comando define el tipo de paquete y la estructura del payload. En el caso que la longitud del payload no sea constante se debe utilizar como referencia el payload length para recibir el paquete sin errores.

4. Detalles de las tareas

1. Tarea de lectura de presión

```
uint32_t Lectura_Presion(){
    uint16_t carotid, femoral;

    // Primero obtengo desde donde voy a obtener la informacion, idealmente deberia ser el mismo perisferico pero no necesariamente.
    uint8_t carotid_source = GetSignalSource(CAROTID);
    uint8_t femoral_source = GetSignalSource(FEMORAL);

#ifdef USE_ADS1192
    // Si alguno de los datos los tengo que leer desde el SPI me tengo que asegurar que tenga un dato disponible para leer
    if((carotid_source == SPI_SOURCE)||(femoral_source==SPI_SOURCE)){
        if(ads1192_interrupt_flag()==0) return 0;
    }
#endif

    /* Obtengo el valor de presion, le envio como parametros la arteria y la fuente de donde leer */
    carotid = GetPressure(CAROTID, carotid_source);                    //Obtengo el dato de presion
    StoreCarotidPressure(carotid, getMeassureMethod(carotid_source));  //Lo guardo en la memoria p

    /* Obtengo el valor de presion, le envio como parametros la arteria  la fuente de donde leer */
    femoral = GetPressure(FEMORAL, femoral_source);                    //Obtengo el dato de presio
    StoreFemoralPressure(femoral, getMeassureMethod(femoral_source));  //Lo guardo en la memoria p

    return 0;
}
```

Obtengo las fuente de donde voy a adquirir los datos

Este bloque solamente es necesario cuando compile usando el ADS1192. Cuando estoy leyendo desde el SPI tengo que esperar el flag de que tengo un dato listo para leer

Leo el valor de presion con la funcion provisa por la HAL Layer y lo guardo con la funcion que empaqueta los datos y los envia

2. Tarea de lectura del sensor PPG

```
uint32_t Lectura_SpO2(){
    static uint16_t initialDelay=0;

    // Planteo un delay inicial para que no se superponga con el envio de datos de la presion
    if(initialDelay < 16) {
        ++initialDelay;
        return 0;
    }

    uint16_t ir_data, red_data;

    // Leo el dato de PPG, en caso que no haya dato disponible o falle algo esta funcion devuelve 0
    if(LeerDatoSpO2(&ir_data, &red_data)){
#ifdef USE_MAX30100
        // Clean last 3 LSB bits
        ir_data &= 0xFFF8;
        red_data &= 0xFFF8;
#endif

        // Invierto la señal asi es comparable con los datos de los sensores tonometricos
        ir_data = 65536 - ir_data;
        red_data = 65536 - red_data;

        // Almaceno el dato y lo acumulo para enviar todo junto
        StoreSpO2Data(ir_data, red_data);
    }
    return 0;
}
```

Hago un delay inicial para que no me colisionen los envíos de datos por el puerto, si bien esto no debería causar problemas pero no esta demás tratar de evitarlos.

Si el sensor utilizado es el MAX30100 tengo que limpiar los ultimos 3 bits del dato

Invierto la señal

Almaceno el dato y lo envió cuando completo el paquete.

3. Tarea de lectura de distancia

```
uint32_t Lectura_Distancia(){
    static uint32_t contadorMuestreo = 0;
    static uint16_t muestras[CANTIDAD_MUESTRAS_DISTANCIA];
    static int16_t lastInserted = -1;

    uint32_t distancia=0;

    // Avanzo el contador, si llego al final vuelvo a empezar.
    // Este contador lo uso como indice para cargar los datos en el buffer para la media movil
    if(++lastInserted == CANTIDAD_MUESTRAS_DISTANCIA) lastInserted = 0;

    // Cargo la medicion en el array
    muestras[lastInserted] = obtenerMedicionDistancia();

    // Calculo el promedio
    for(int16_t i=0; i<CANTIDAD_MUESTRAS_DISTANCIA; i++)
        distancia += muestras[i];
    distancia = distancia / CANTIDAD_MUESTRAS_DISTANCIA;

    //distancia *= FACTOR_DE_CONVERSION_DIST;

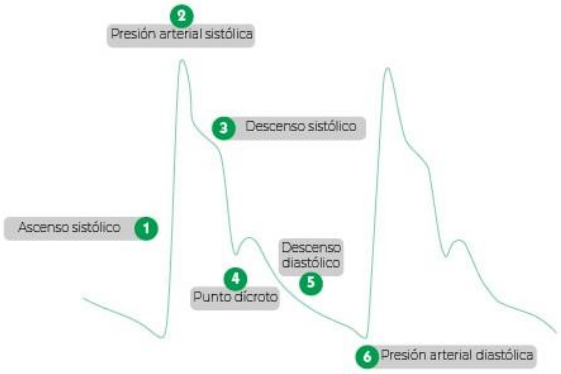
    lastDistance = ((float)distancia/4095.0)*(DISTANCIA_MAXIMA-DISTANCIA_MINIMA);

    lastDistance += DISTANCIA_MINIMA;
    // Incremento el contador para enviar el dato al cliente
    if(++contadorMuestreo >= PERIODO_REPORTE_DISTANCIA_ms){
        // Reseteo el contador
        contadorMuestreo = 0;
        // Lo envio
        SendDistance(distancia);
    }

    return 0;
}
```

4. Tarea de análisis de datos

La tarea dispone de dos estructuras de datos que son compartidas con las funciones que empaquetan los datos de la señal de presión. Esta función cada vez que tiene un dato nuevo se lo agrega a esta memoria compartida. La tarea de análisis de datos funciona de manera no apropiativa, es decir, no tiene el mismo privilegio de uso de CPU que las demás tareas, e incluso comparte el excedente de CPU con las demás tareas no apropiativas. Es por esto que el ciclo de ejecución de esta tarea no es constante, cuando el scheduler le asigne recursos podrá ejecutarse. Dentro de la estructura compartida tiene el índice del ultimo dato agregado a la estructura y el ultimo dato procesado, de esta manera se da cuenta que tiene un dato nuevo. Como ambas estructuras se analizan de la misma manera se implementó una función aparte para el análisis de la estructura. Esta función es la llamada `void analisis_de_datos (OS_SharedMemory_Struct *pSharedMemory)` que puede recibir tanto datos de presión carótida como femoral ya que el análisis es el mismo.



El objetivo de esta función es detectar el ascenso sistólico, de esta forma podemos determinar el pie de onda (en el código es referenciado como pico diastólico) y el pico sistólico que corresponde a la presión arterial sistólica. Con los picos sistólicos de la arteria carótida podemos determinar el ritmo cardiaco y con el desfasaje temporal de los pies de onda podemos calcular la velocidad de onda de pulso o al menos presentar los datos para que puedan ser calculados en el cliente.

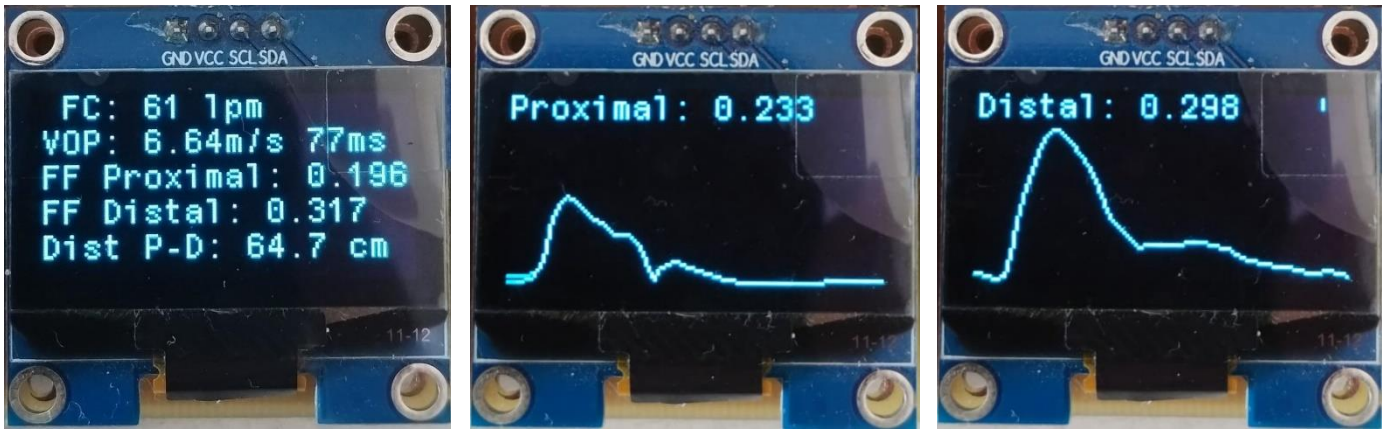
5. Tarea de visualización

La visualización se hace mediante una pantalla OLED de 64x128 pixeles. El controlador que maneja el display tiene un buffer interno donde cada bit representa un píxel. Los cambios en la pantalla se realizan sobre este buffer. Luego, cada cierto tiempo, el buffer es enviado entero hacia la pantalla. Este envió se realiza mediante DMA por lo tanto no ocupa tiempo del procesador. El tiempo de actualización de la pantalla también se puede configurar en el archivo config.h

La visualización incluye tres pantallas, en la primera se puede visualizar la medición de los latidos por minutos, el vop Delay, el factor de forma de ambas señales y la distancia entre los sensores que se fija mediante el potenciómetro integrado. En la segunda pantalla se visualiza una captura del ultimo ciclo de la señal de la arteria carótida y luego en una tercera pantalla se visualiza una captura del ultimo ciclo de la señal de la arteria femoral. En el caso que la amplitud de la señal este por encima de las 2000 cuentas del adc en estas pantallas además se visualiza el facto de forma de la señal.

Para el calculo del factor de forma se utilizó el siguiente calculo [4]:

> Mean arterial pressure (MAP) is often estimated from cuff systolic (S) and diastolic (D) blood pressure (BP) using a fixed arterial form factor (FF, usually 0.33). If MAP is measured directly, a true FF can be calculated: FF = [MAP–DBP]/[SBP–DBP]. Because waveform shapes vary, true FF should also vary and MAP accuracy will be affected. We studied factors affecting FF using radial tonography



Los tiempos de cada pantalla pueden ser ajustados desde el archivo config.h, los defines se ajustan en milisegundos.

5. Uso de la memoria del microcontrolador

Por último, luego de tener todo el programa subido al microcontrolador este es el uso final de los recursos del procesador.

uFISIO.elf - /uFISIO/Debug - May 23, 2023, 9:42:19 PM

Memory Regions | Memory Details

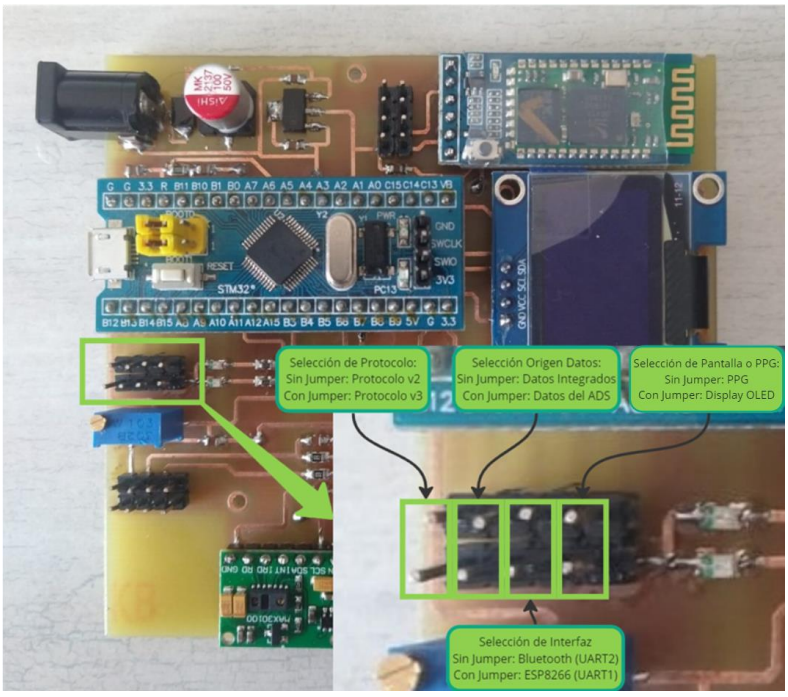
Region	Start address	End address	Size	Free	Used	Usage (%)
RAM	0x20000000	0x20004fff	20 KB	3.71 KB	16.29 KB	81.45%
FLASH	0x08000000	0x0800ffff	64 KB	1.79 KB	62.21 KB	97.21%

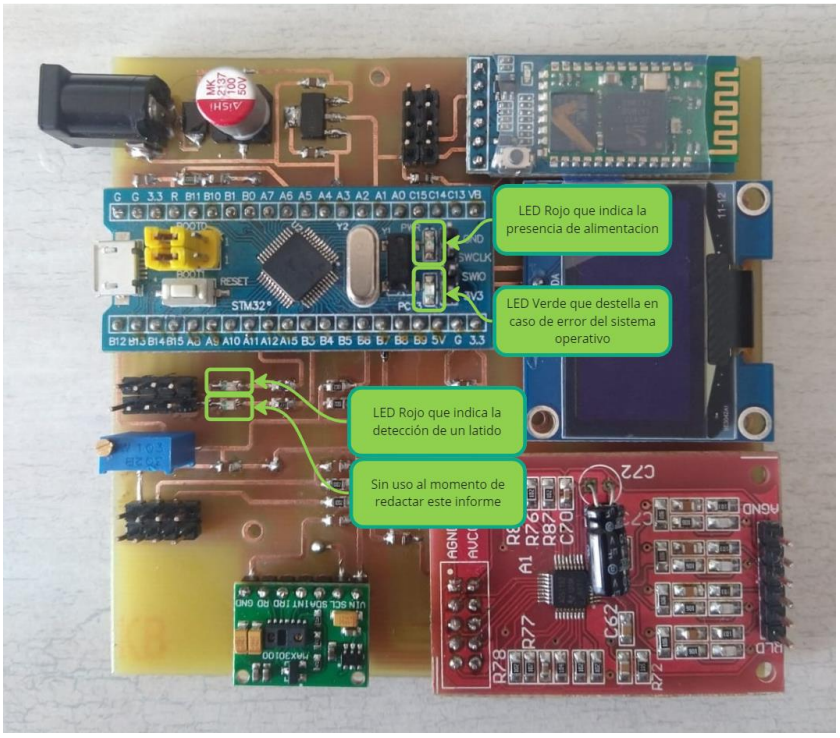
6. Clientes de Ejemplo

Si bien el desarrollo de clientes de visualización no es parte de este proyecto era necesario tener alguna herramienta para probar el desempeño de la plataforma. Por este motivo se desarrolló un cliente en Python con capacidad de recibir e interpretar todos los mensajes enviados por la plataforma. También se desarrollaron algunas clases en C++ para que en un futuro se pueda implementar un cliente en QT5. Se recomienda mirar el archivo test.py dentro de la carpeta “Clientes\Python\” donde está un ejemplo sencillo de implementación de las clases.

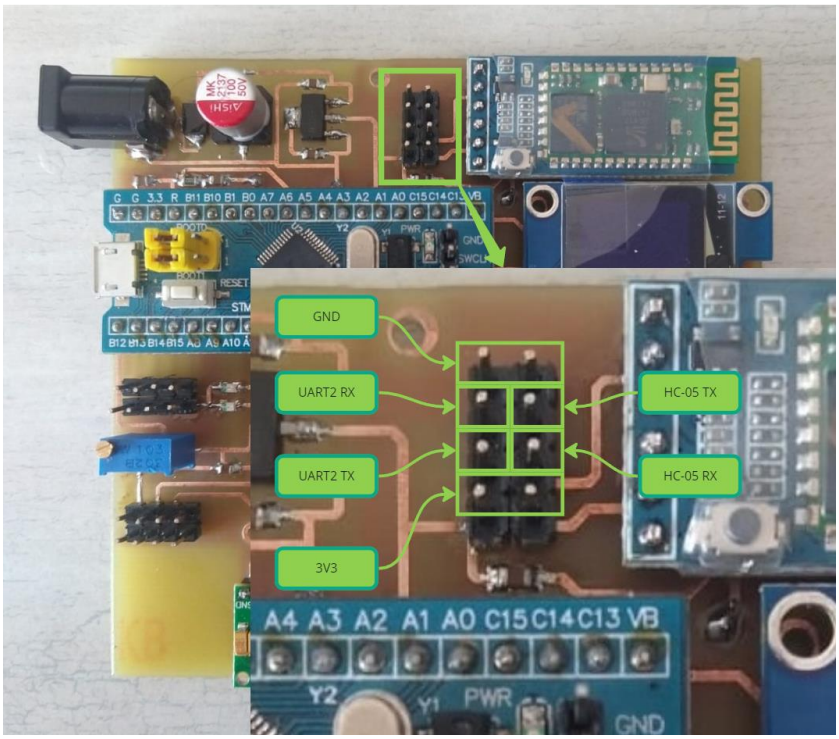
4. Resultados

1. Detalle de la placa

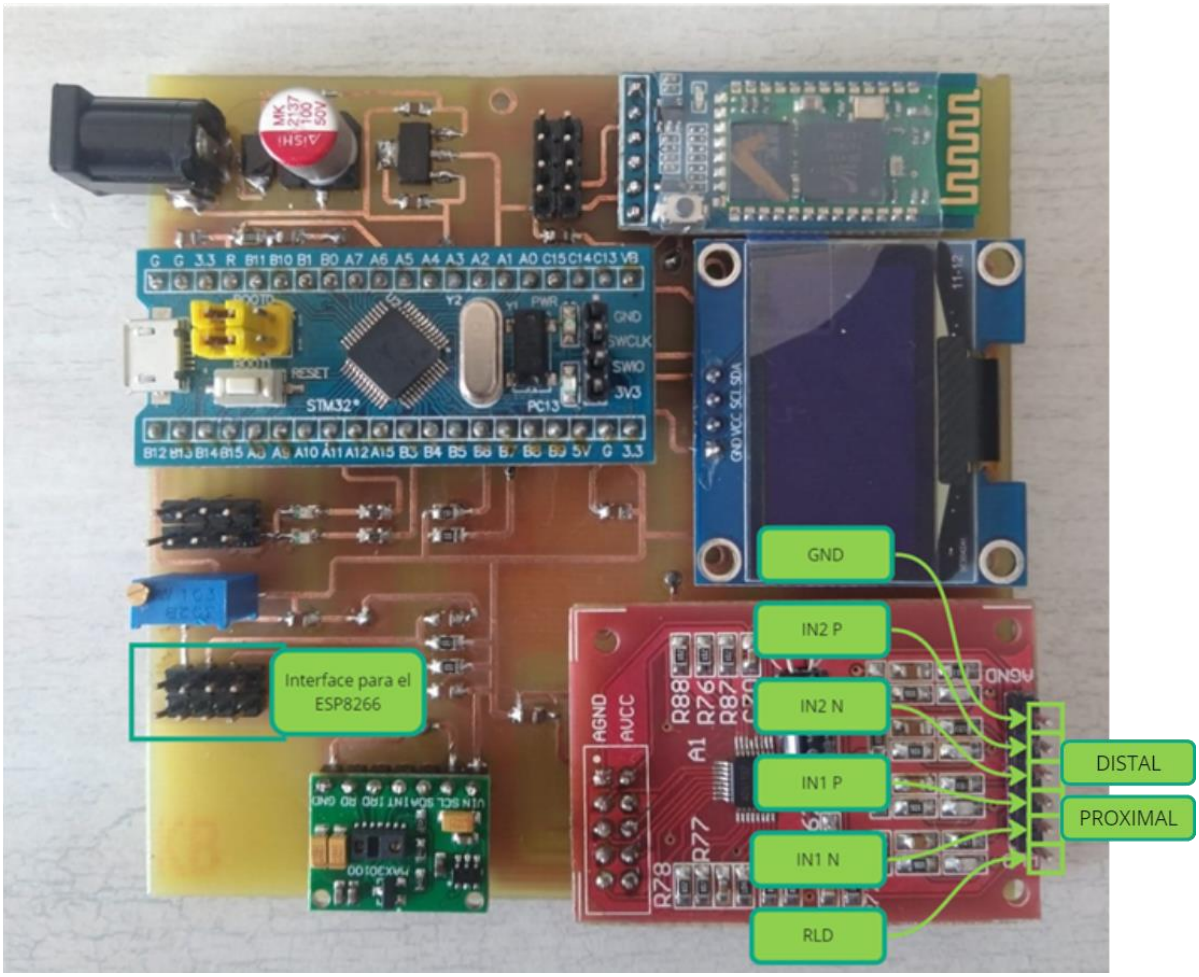




Detalle de Puerto UART2 (Bluetooth)



Para el uso del módulo bluetooth hay que colocar un jumper entre las líneas UART2 RX y HC-05 TX y otro entre las líneas UART2 TX y HC-05. Además, hay que seleccionar que los datos se transmitan por el puerto UART2 dejando sin colocar el Jumper de la entrada digital B14. El objetivo de esta conexión es que se tenga pines disponibles para poder conectar un adaptador USB-TTL para configurar el módulo bluetooth.

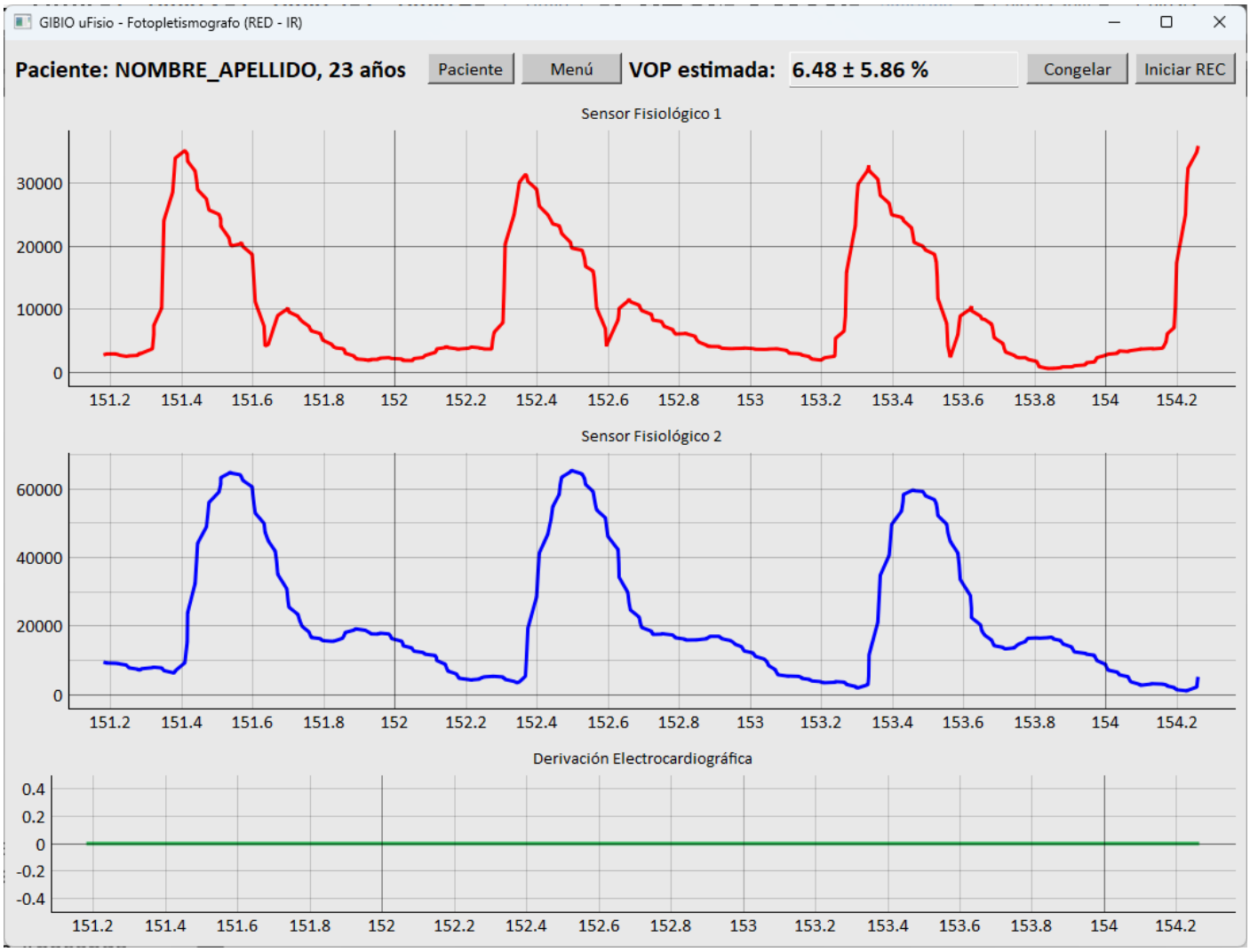


2. Utilizando el Software provisto por el GIBIO

Dado que no se tenían sensores piezoeléctricos para hacer las mediciones con señales reales se utilizó los datos incorporada en la memoria del microcontrolador para observar la visualización de la señal.

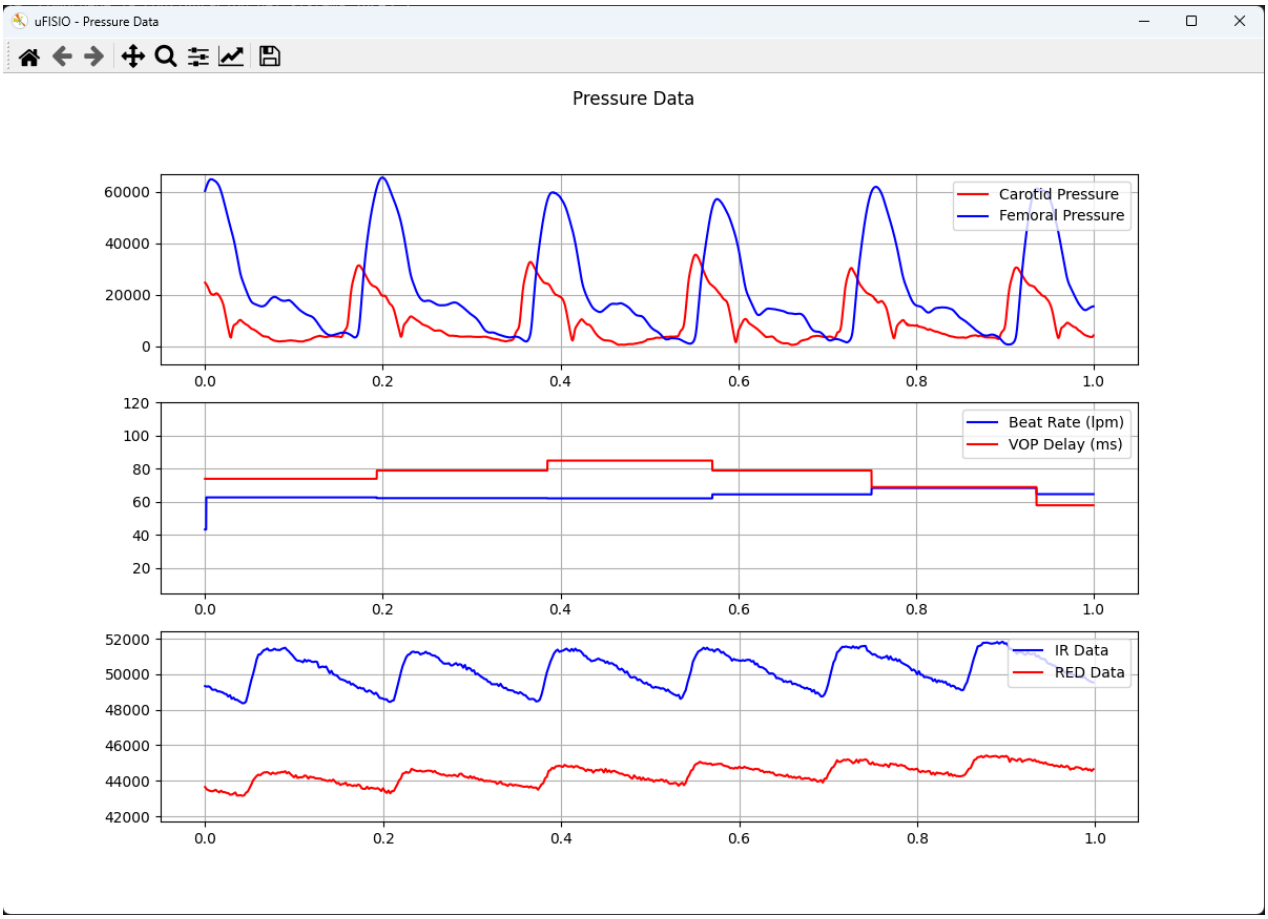
Para la ejecución del programa provisto por el GIBIO se instaló el framework de trabajo Anaconda y se instalaron las dependencias según expresa la documentación asociada al proyecto.

Las pruebas con el cliente fueron satisfactorias, se pudo visualizar la señal sin problema y el programa arrojo un cálculo de la VOP_{CF} de 6.48 m/s



3. Utilizando el cliente de pruebas

El cliente de pruebas muestra en tiempo real todas las señales recibidas desde la plataforma. Dada su construcción simple en un solo hilo de ejecución no se puede mostrar la señal a la misma velocidad a la que se recibe, de cualquier manera, es suficiente y cumple con las expectativas




```
[   CPU USAGE    ] Best time: 44 Worst time: 519
     00.16%       Task: ck_pend  Run time: 141906 Best time: 1 Worst time: 11
     01.74%       Task: l_pres   Run time: 1564440 Best time: 13 Worst time: 143
     02.55%       Task: l_spo2   Run time: 2291160 Best time: 2 Worst time: 352
     02.85%       Task: l_dist   Run time: 2566192 Best time: 28 Worst time: 52
     46.14%       Task: t_visua  Run time: 41522668 Best time: 0 Worst time: 0
     46.57%       Task: t_anali  Run time: 41913634 Best time: 0 Worst time: 0
     00.00%       Task:          Run time: 0 Best time: 0 Worst time: 0
     00.00%       Task:          Run time: 0 Best time: 0 Worst time: 0
```

En el primer grafico se visualiza la señal de presión carótida y femoral integrada en memoria, en la gráfica del medio se representa la información del ritmo cardiaco y el desfasaje temporal entre el pie de onda de la señal pulsátil de la arteria carótida y la arteria femoral y en la gráfica de abajo vemos los datos obtenidos del sensor fotopletismográfico. En este caso, se apoyó el dedo índice sobre el sensor por lo cual la medición corresponde a un caso real.

Con el dato de desfasaje temporal podemos calcular la VOP_{CF} utilizando la siguiente ecuación:

$$VOP_{CF} = 0.8 * \frac{dist(m)}{desf(s)} = 0.8 * \frac{0.60\ m}{0.073\ seg} = 6.57\ m/s$$

El cálculo se realizó tomando el promedio de VOP Delay y una distancia carótida-femoral de 60cm que es el mismo valor que se ingresó en la prueba anterior. Como se puede observar el valor se aproxima bastante al valor informado por la aplicación de referencia.

5. Conclusiones

El desarrollo de la plataforma cumplió con las expectativas propuestas. Se implementó un sistema operativo escalable que podría ser utilizado en futuros proyectos del GIBIO, también se desarrolló un protocolo universal de comunicación el cual prevé control de integridad de datos, perdida de paquetes, incluso sobre el mismo protocolo se puede implementar un sistema de acknowledge y retransmisión de paquetes perdidos. Además, se implementó en el dispositivo una rutina de análisis de señales en tiempo real para calcular el ritmo cardiaco mediante la detección del pico sistólico en la señal pulsátil de la arteria carótida y se calculó el desfasaje temporal entre ambas señales de presión para calcular la VOP_{CF}. Si bien el cálculo final debe hacerse en el cliente, ya que se necesitan algunos datos adicionales del paciente, la parte que requiere más recursos y conocimientos específicos ya se encuentra desarrollada en el microcontrolador. Se comparó el resultado obtenido mediante este método con el ofrecido por la aplicación provista por el GIBIO. El reporte de la plataforma indica una VOP_{CF} de 6.57 m/s mientras que la aplicación reporta 6.45 m/s sobre la misma señal. Por lo tanto, el resultado es satisfactorio en comparación con la aplicación de referencia.

6. Referencias

[1] L. M. Van Bortel, T. De Backer, and P. Segers. 2016. “Standardization of Arterial Stiffness Measurements Make Them Ready for Use in Clinical Practice.”

[2] J. Blacher, R. Asmar, S. Djane, G. M. London, and M. E. Safar. 1999. “Aortic Pulse Wave Velocity as a Marker of Cardiovascular Risk in Hypertensive Patients.”

[3] M. De Luca, L. J. Cymberknop, N. Bortoni, P. Vassena, S. Mac Mullen, D. Cafa, R. Armentano. “uFISIO-VOP: Implementación de un Sistema Modular para la Evaluación de la Rigidez Arterial en Entornos Clínicos”. XXII Congreso Argentino de Bioingeniería (2020)

[4] Joseph L. Izzo, Debduti Mukhopadhyay, Sagar Nagpal and Peter Osmond “Impact of arterial location, pressure wave indicators, and measurement devices on arterial form factor and mean and central arterial pressure”, December 2022